

DR-Platform v 1

Kubernetes 다중 클러스터 재해복구(DR) 및 Failback 자동화 플랫폼

최종 보고서 | Final Project Report

1 조

과목명	클라우드 가상화기술
팀명	1 조
프로젝트명	DR-Platform
문서 버전	v3.1 (최종보고서)
대상 브랜치	main
작성일	2026. 06. 14

Executive Summary

문제 (Problem)

Kubernetes 기반 서비스는 클러스터 장애, 네트워크 단절, 스토리지 장애가 발생했을 때 백업·복원 절차가 여러 도구와 권한 경계에 흩어져 있다. 운영자는 장애 대피(Failover)와 원상 복구(Failback)를 같은 흐름에서 추적하기 어렵고, 특히 메인망과 임시망이 분리된 환경에서는 플랫폼이 모든 클러스터에 직접 접근하기 어렵다.

해결 방식 (Solution)

DR-Platform은 Cloud K8s와 Edge K3s를 하나의 재해복구 흐름으로 연결한다. dr-agent가 각 클러스터의 상태를 Heartbeat Push 방식으로 전달하고, Velero/MinIO 기반 백업 저장소와 복구 정책 엔진을 통해 Failover 및 Failback 절차를 자동화하였다. 단, 메인망 복원은 보안과 권한 격리를 위해 관리자가 다운로드한 스크립트를 직접 실행하는 구조로 조정하였다.

결과 (Result)

중간보고서의 Phase 1 및 Phase 1.5 범위는 구현 완료되었고, Phase 2 항목 중 외부 공개 URL(zrok) 기능은 앞당겨 반영되었다. 실제 Node.js 백엔드와 Postgres DB를 배포하여 Cloud K8s 장애 발생, Edge K3s 대피 운영, K3s 최신 데이터 백업, K8s 원상 복구 스크립트 실행까지 End-to-End 시나리오를 검증하였다.

진척도 요약

페이지	계획 범위	달성 상태	비고
Phase 1 (MVP / P0)	다중 클러스터 모니터링, 수동 백업/복원, 대시보드 UI	완료	dr-agent 및 관제 대시보드 연동 완료
Phase 1.5 (P1)	Failback 자동화 로직, 복구 정책(Policy), drctl CLI	완료	8단계 온보딩 및 복구 스크립트 생성 로직 구현
Phase 2 (고도화)	터널링(zrok) 외부 노출, 스토리지 자동 프로 비저닝	부분	공개 URL 구현 완료, 스토리지 자동화는 이연

목차

Executive Summary

1. 보고서 개요
2. 개발 로드맵 진척도 평가
3. 중간보고서 대비 주요 변경점
4. 구현 아키텍처
5. 핵심 구현 기능
6. End-to-End 배포 검증 결과
7. 기능 목록 최종 현황 (MoSCoW 대비)
8. 트러블 슈팅 및 문제 해결
9. 한계 및 향후 과제
10. 결론

1. 보고서 개요

1.1 목적

본 최종보고서는 중간보고서에서 설계한 DR-Platform 아키텍처와 개발 로드맵이 실제 코드로 어떻게 구현되었는지를 검증하고, 설계 대비 변경된 사항을 정리한다. 중간보고서가 “무엇을 만들 것인가(설계)”였다면, 본 보고서는 “무엇을 만들었는가(구현 결과)”와 “왜 설계와 달라졌는가(변경 근거)”를 시스템 및 코드 레벨에서 기술한다.

검증 핵심 산출물은 Cloud K8s(메인망) 및 Edge K3s(임시망) 멀티 클러스터 환경을 구축하고, 실제 서비스 장애 시뮬레이션을 통해 Failover(장애 대피)부터 Failback(원상 복구)까지의 End-to-End 파이프라인이 정상 동작함을 확인한 것이다.

1.2 전체 진척도 한눈에 보기

페이지	계획 범위	달성 상태	비고
Phase 1 (MVP / P0)	다중 클러스터 모니터링, 수동 백업/복원, 대시보드 UI	완료	dr-agent 및 관제 대시보드 연동 완료
Phase 1.5 (P1)	Failback 자동화 로직, 복구 정책(Policy), drctl CLI	완료	8단계 온보딩 및 복구 스크립트 생성 로직 구현
Phase 2 (고도화)	터널링(zrok) 외부 노출, 스토리지 자동 프로비저닝	부분	터널링 및 공개 URL 구현 완료, 스토리지는 이연

2. 개발 로드맵 진척도 평가

2.1 Phase 1 (MVP) — 완료

마일스톤: “다중 클러스터 상태 수집 및 Velero 백업 연동”

- dr-agent 구현: 대상 클러스터에 배포되어 워크로드 상태 및 백업 현황을 수집하는 에이전트(Heartbeat 방식).
- 중앙 관제 대시보드: React 기반 UI를 통해 여러 클러스터의 상태 및 경고(Alert)를 시각화.
- 백업/복원 기초 구현: Velero를 이용한 네임스페이스 단위의 K8s 리소스 백업 및 S3(MinIO) 업로드 기능.

2.2 Phase 1.5 (P1) — 완료

마일스톤: “Failback 파이프라인 자동화 및 CLI 도구 배포”

- Failover & Failback 엔진: 장애 발생 시 Edge K3s로 복원하고, 복구 완료 후 K3s의 최신 데이터를 다시 K8s로 돌려보내는 자동화 파이프라인.
- 복구 스크립트 발급기: Failback 진행 시 플랫폼이 자동 백업을 수행하고 사용자가 실행할 수 있는 restore-*.sh 다운로드 기능 구현.
- drctl CLI: 터미널에서 플랫폼 연동(init) 및 복구 정책(policy set)을 손쉽게 설정할 수 있는 CLI 도구 배포.

2.3 Phase 2 (고도화) — 부분 구현

Phase 2 과제 중 외부망 접근 제약 해소를 위한 zrok 기반 터널링과 공개 URL 지원을 앞당겨 구현하였다. 반면 클라우드 네이티브 스토리지(EBS, EFS 등) 자동 프로비저닝은 로컬 MinIO 기반의 인프라 제약으로 인해 향후 과제로 이연하였다.

3. 중간보고서 대비 주요 변경점

#	구분	중간보고서(설계)	최종(구현)	변경 근거
1	Failback 실행	플랫폼이 타겟 클러스터에 자동 복원 명령	사용자 수동 스크립트 실행	K8s 메인망에 데이터를 덮어쓰는 작업의 치명성을 고려하여 망 분리 및 권한 격리 원칙에 맞게 변경.
2	상태 동기화	플랫폼이 클러스터를 Polling	Agent Heartbeat Push	사설망 클러스터 접근 불가 문제를 해결하기 위해 에이전트가 중앙 서버로 상태를 Push.
3	스토리지 격리	네임스페이스 기반 논리 격리	Velero Prefix 기반 격리	단일 MinIO 공유 환경에서 prefix=<CLUSTER_ID>/ 구조로 사용자별 백업 데이터를 분리.
4	UI/UX 온보딩	통합 스크립트 제공	8단계 상세 가이드라인	데모 투명성 및 기술적 신뢰도 확보를 위해 내부 과정을 단계별로 노출.
5	모니터링 스택	Zabbix	Prometheus	K8s 환경에 더 최적화된 성능과 연동성을 제공하고 설정 복잡도를 낮춤.

4. 구현 아키텍처

4.1 4계층 구조

- Presentation (React 18): 클러스터 헬스 체크, Failback 마법사, 알림 및 토폴로지 시각화 패널 제공.
- Business Logic (Node.js/Express): Agent Heartbeat 수신 엔진, 복구 정책 관리, Firing/Resolved 알람 상태 전이 엔진(server.mjs).
- Integration (Velero & CLI): S3(MinIO) 스토리지 연동, 백업/복원 REST API 연동, drctl을 통한 설정 자동화.

- K8s Cluster: Cloud K8s(Primary / 메인망)와 Edge K3s(Recovery / 임시망)로 구성.

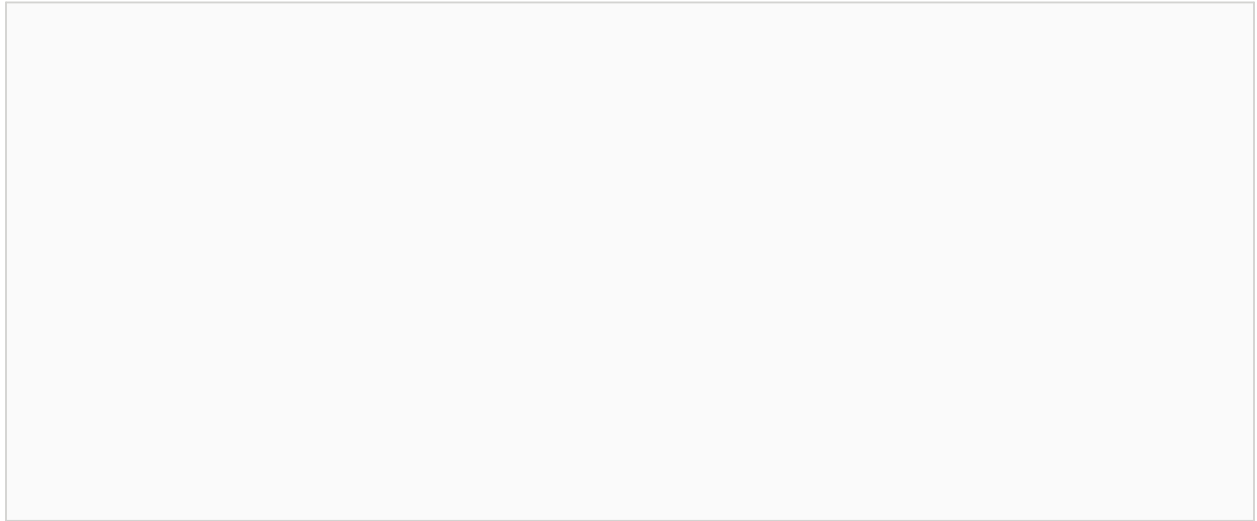


그림 1. DR-Platform 구현 아키텍처

4.2 전역 상태 관리 및 이벤트 엔진

Heartbeat 기반 알람 해소 로직: K3s 클러스터 복구 완료 후, 정상 동작 중인 네임스페이스 정보를 수신하면 과거 메인망에서 발생했던 Firing 상태의 알람을 Resolved로 자동 전환하는 이벤트 엔진을 server.mjs에 구현하였다.

비동기 Operation 큐: 백업/복원과 같이 장시간이 소요되는 작업은 Operation 큐로 관리하고 프론트엔드는 이를 폴링하여 진행률(%)을 렌더링한다.

5. 핵심 구현 기능

5.1 Heartbeat 기반 다중 클러스터 동기화 (dr-agent)

- 통신 아키텍처: 플랫폼 서버가 사설망에 갇힌 클러스터에 직접 접근할 수 없는 네트워크 구조적 한계를 극복하기 위해, 에이전트가 중앙 서버로 주도적으로 데이터를 보내는 Heartbeat Push 모델을 채택하였다.
- 지표 수집: Helm으로 배포된 dr-agent는 30초 단위로 클러스터의 노드 상태, 파드 레플리카, Velero 백업 및 복원 이력을 server.mjs의 receiveAgentHeartbeat API로 전송한다.
- FSM 기반 알람 자동 해소: 장애 발생 시 생성된 Firing 상태의 알람은 클러스터가 복구되어 워크로드가 Running 상태로 보고되면 서버 측 이벤트 엔진이 이를 인지하여 Resolved 상태로 전이한다.

5.2 Policy-as-Code 기반 SLA 관리 엔진

- CLI 기반 선언적 관리: drctl policy set 명령어를 통해 각 네임스페이스별 RTO, RPO, 서비스 등급(Tier)을 선언적으로 정의한다.

- Backup Freshness 모니터링: 대시보드는 등록된 Policy(RPO)와 가장 최근 성공 백업 완료 시간을 비교하여 RPO 초과 시 UI 경고를 노출한다.

5.3 Failback 자동화 파이프라인 및 복구 스크립트 발급

- 양방향 Failback 오케스트레이션: 대시보드에서 Failback을 트리거하면 서버가 K3s 에이전트의 큐에 BackupCommand를 삽입하고, 에이전트가 K3s 상에서 Velero Backup을 자동 실행한다.
- 비동기 상태 폴링: 프론트엔드는 failback 엔드포인트를 폴링하며 Queued → Submitted → InProgress → Completed 전이와 진행률을 실시간 렌더링한다.
- 권한 분리형 스크립트 제공: 백업 완료 후 restore-`<backupName>.sh`를 생성하고, 관리자가 메인망에서 직접 실행하도록 하여 보안 및 권한 격리 원칙을 준수하였다.

5.4 Velero Prefix 기반 멀티테넌시 및 데이터 격리

- 스토리지 통합 및 격리: 여러 테넌트가 백업 저장소로 단일 S3(MinIO) 버킷을 공유하는 환경을 구성하였다.
- 디렉토리 논리적 격리: velero install 단계에서 --backup-location-config prefix=`<CLUSTER_ID>`/ 옵션을 강제하여 각 클러스터 백업 데이터가 고유 디렉토리에 저장되도록 설계하였다.
- 보안성 확보: A 클러스터의 Velero는 A/ 디렉토리 내의 백업 객체만 인지하므로 다른 사용자의 데이터를 복원하거나 덮어쓰는 논리적 데이터 오염 사고를 차단한다.

6. End-to-End 배포 검증 결과

실제 Node.js 백엔드 앱과 Postgres DB를 K8s에 배포하여 재해복구 시나리오를 검증하였다.

1. 배포 전 초기 상태: Cloud K8s에서 서비스 정상 동작 및 주기적 백업 진행.
2. Stage 1 (장애 발생 및 Failover): Cloud K8s의 장애를 시뮬레이션하고, 대피소인 Edge K3s에서 이전 백업본을 가져와 시스템 복원.
3. Stage 2 (K3s 임시 운영): K3s 환경에서 서비스가 구동되며 새로운 게시글 및 DB 데이터가 누적됨.
4. Stage 3 (Failback 버튼 클릭): 대시보드에서 Fail back to primary cluster를 실행하고 플랫폼이 K3s의 최신 상태를 MinIO에 백업 완료.
5. 최종 검증: 다운로드받은 .sh 스크립트를 복구된 K8s 터미널에서 실행하여 K3s에서 생성했던 최신 게시글 데이터까지 유실 없이 원상 복구됨을 확인.

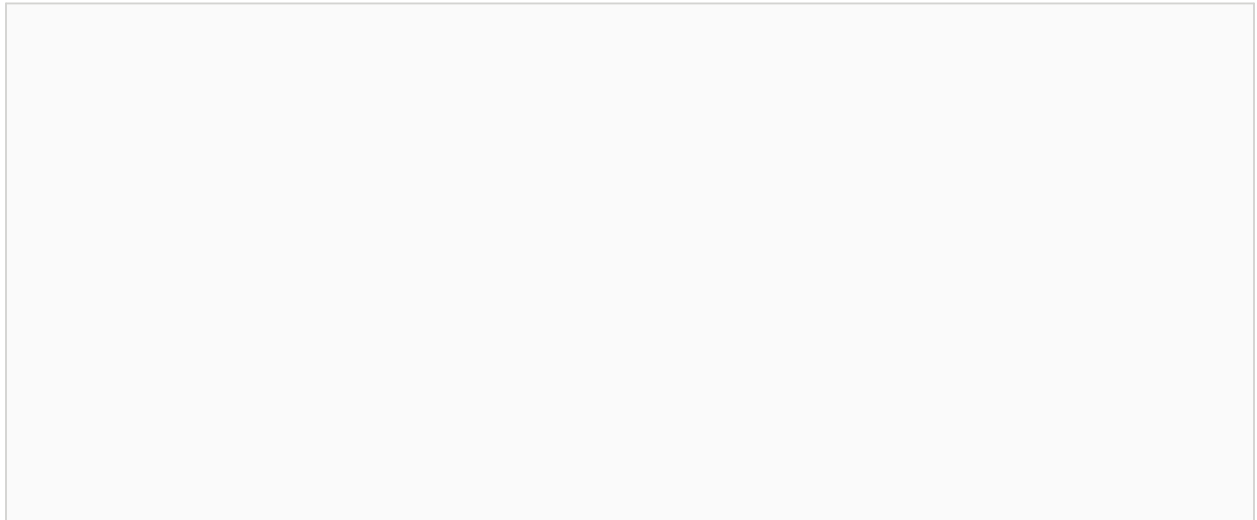


그림 2. Failover 및 Failback 실패포 검증 화면

7. 기능 목록 최종 현황 (MoSCoW 대비)

ID	기능명	MoSCoW	상태
F-01	drctl CLI 연동 도구	Must	완료
F-02	dr-agent Helm Chart 패키징	Must	완료
F-03	대시보드 상태 모니터링 패널	Must	완료
F-04	복구 정책(Policy-as-Code) 관리	Must	완료
F-05	Failover(K3s 대피) 파이프라인	Must	완료
F-06	Failback 스크립트 자동 생성기	Should	완료
F-07	멀티테넌시 스토리지 버킷 격리	Should	완료
F-08	8단계 온보딩 UI 마법사	Should	완료
F-09	터널링(zrok) 기반 공개 URL 제공	Could	완료 (앞당김)
F-10	타겟 클러스터 자동 Failback	Could	보류 (수동 스크립트로 대체)
F-11	클라우드 스토리지 자동 프로비저닝	Could	이연 (Phase 2)

Must/Should 기능은 전량 완료되었으며, Could에 해당하던 외부 URL 노출 기능을 앞당겨 구현하였다. 치명적 보안 위협을 방지하기 위해 완전 자동 Failback(F-10) 설계는 보류하고 수동 스크립트 발급 구조로 변경하였다.

8. 트러블 슈팅 및 문제 해결

8.1 관리형 클러스터 권한 누락 및 UI 블랭크 이슈

증상: 사용자 토큰에서 중앙 클러스터 접근 권한이 누락되면서 대시보드 렌더링에 필요한 필수 상태를 가져오지 못해 UI가 깨지거나 멈춤.

원인: UI 컴포넌트가 중앙 클러스터 상태에 의존했으나, 백엔드 API가 접근을 엄격하게 404 에러로 차단.

해결: tokens.json에 필수 클러스터 권한을 복구하고, App.jsx에서 중앙 클러스터가 좌측 사이드바에 노출되지 않도록 필터링 로직을 추가.

8.2 가상(Mock) 환경 SSH 실행 오류로 인한 API Error 도배

증상: UI 상단 메트릭 패널 전체가 빨간색 API error로 도배되고 상단 타이틀이 cloud-primary로 고정됨.

원인: 프론트엔드 useEffect 훅이 초기 렌더링 시 사용자의 test 클러스터 대신 첫 번째 항목인 cloud-primary를 강제로 선택.

해결: 활성화된 클러스터가 없을 경우 중앙 관리형 클러스터를 건너뛰고 사용자의 실제 클러스터(test)를 우선 자동 선택하도록 수정.

8.3 Helm Repository 업데이트 JSON Unmarshaling 에러

증상: helm repo update 실행 시 cannot unmarshal string into Go value of type repo.IndexFile 에러 발생.

원인: 외부 터널 주소가 백엔드 API 서버가 아닌 Vite 개발 서버를 직접 가리켜 Helm이 JSON이 아닌 HTML 응답을 수신.

해결: vite.config.js에 /api 요청을 백엔드 서버로 포워딩하는 프록시 설정을 추가.

8.4 Agent 설치 후 대시보드 맵에서 클러스터 에러 지속

증상: DR Agent를 정상적으로 Helm 설치했음에도 대시보드에서 녹색(Safe)이 아니라 계속 에러 상태로 노출.

원인: 가이드의 가짜 토큰이 그대로 지정되어 백엔드가 Heartbeat를 차단했고, Proxy 연동 누락으로 외부 터널 신호가 백엔드까지 도달하지 못함.

해결: 실제 토큰과 네임스페이스 옵션을 추가하여 Helm 배포를 재진행하고, 프론트-백엔드 Proxy 서버를 재시작.

9. 한계 및 향후 과제

1. 완전 자동화된 Failback의 부재: 현재 아키텍처는 권한 제약을 피하기 위해 사용자가 직접 메인망에서 스크립트를 실행해야 한다. 향후 ArgoCD 등을 활용한 GitOps 방식과 연계하면 수동 개입 없는 자동 복구 파이프라인으로 고도화할 수 있다.
2. 스토리지 벤더 종속성 최소화: 현재 테스트는 S3 호환 로컬 스토리지인 MinIO 환경에 맞춰져 있다. 프로덕션 배포를 위해 AWS EBS, EFS 및 GCP 영구 디스크(PD)의 볼륨 스냅샷 기능까지 포괄하는 확장이 필요하다.
3. 원클릭 설치 스크립트 지원: 투명성을 위한 8단계 설치 과정이 온보딩 장벽이 될 수 있다. 향후 curl 기반 원클릭 스크립트 버전을 추가로 지원할 계획이다.

10. 결론

DR-Platform은 중간보고서에서 설계한 K8s 다중 클러스터 재해복구 파이프라인을 성공적으로 구현하였다. 핵심 마일스톤인 “장애 발생(K8s) → 대피소 복원(K3s) → 최신 데이터 백업 → 원상 복구 스크립트(.sh) 실행(K8s)”으로 이어지는 End-to-End 재해복구 사이클이 검증되었다.

설계 대비 일부 변경(스크립트 발급 방식, Heartbeat 도입 등)은 모두 시스템 보안 및 물리적 망 분리 환경을 고려한 현실적 판단에 따른 것이다. 결과적으로 플랫폼의 안정성과 데이터 격리성을 강화하였으며, 이연된 스토리지 프로비저닝 및 GitOps 연계 과제는 상용화를 위한 향후 고도화 목표로 남는다.